
Empirical Demonstration of Stock Paper Trading for Financial Reinforcement Learning

Xiao-Yang Liu and Ziyi Xia
Columbia University

Abstract

Deep reinforcement learning (DRL) has shown great potential in financial tasks. However, existing works optimistically reported profitable results through backtesting, suffering the *look-ahead bias* and *overfitting* issues. Therefore, these promising research results do not necessarily lead to a good performance in real-world markets. In this paper, we provide the first empirical demonstration of the stock paper trading task using deep reinforcement learning. First, we employ a "training-validation-trading" pipeline where an agent always learns the latest market daily data; meanwhile, the pipeline helps avoid information leakage. Then, we show that the trained DRL agent outperforms a conventional machine learning method (random forest) and the Dow Jones Industrial Average (DJIA) index during stock paper trading. Our codes are available online at: <https://github.com/AI4Finance-Foundation/FinRL-Meta>

1 Introduction

Deep reinforcement learning (DRL) has shown great potential in financial tasks. Many research papers have applied DRL algorithms to various applications, including portfolio management [10], trade execution, option pricing and hedging [4], market making, smart order routing, and robo-advising [8].

However, existing works optimistically reported profitable results through *backtesting*. Such backtesting results may experience two major issues, look-ahead bias and overfitting. A backtest is a "historical simulation of how a strategy would have performed should it have deployed over a past time period" [5]. The first issue of backtesting is look-ahead bias, usually caused by information leakage. Even though historical data are often separated into in-sample set (often used for training) and out-of-sample set (often used for testing), information leakage could still happen because people may somewhat get the information of the market trend. Influence of this issue can be significant since it can cause optimistic backtesting results. Another issue is overfitting, which occurs when a trading agent fits the historical training data to a harmful extent. The problem is that the agent's performance may look promising on papers (using backtesting) but is disappointing in practice [1]. Due to the above factors, seemingly promising backtest results do not necessarily lead to profits in real-world markets.

In this paper, we present the first empirical demonstration of stock paper trading for financial reinforcement learning, which is one step closer to real-world stock trading than backtesting. In particular, we show that a trained DRL agent makes profits in stock paper trading. As shown in Fig. 1, the major difference between backtesting and proposed paper trading is that backtesting may be performed multiple times (by replaying historical market data), while stock paper trading is strictly required to be carried out **once** in real-time on the real-world market.

First, we model the paper trading task as a Markov process (MDP) and describe the Proximal Policy Optimization algorithm (PPO) [19]. Then, we employ a "training-validation-trading" pipeline which helps avoid using future information. Finally, we show that our trained DRL agent achieves higher

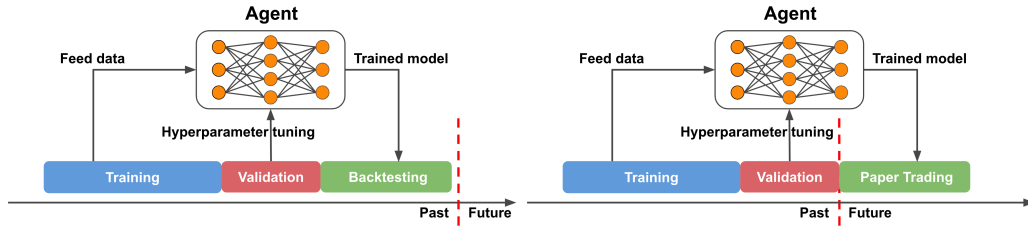


Figure 1: Comparison between the pipelines of backtesting (left) and proposed paper trading (right).

return than a conventional machine learning method (random forest [2]) and the Dow Jones Industrial Average (DJIA) index.

The remainder of this paper is organized as follows. Section 2 presents existing works. Section 3 describes a stock trading task and a pipeline for our stock paper trading task. Section 4 describes the performance evaluations, including data preprocessing, experimental settings, and trading results. Section 5 concludes our work and discusses future work.

2 Related Works

With the development of FinTech, machine learning is widely adopted in finance. In particular, deep reinforcement learning (DRL) is attracting hot attention [6][5]. We will discuss related works that inspire us to provide an empirical demonstration of stock paper trading for financial reinforcement learning.

There are a few methods applying conventional machine learning to paper trading. Ugur *et al.* [7] used CNN to predict stock price movement and evaluated the performance during paper trading. In [20], Sun *et al.* [20] used LSTM to make prediction and also tested the result through paper trading. Those works used paper trading or in addition of backtesting to show their validity.

Existing works on financial reinforcement learning used backtesting, which can be classified into the following three applications:

- For stock trading, Xiong *et al.* [13] gave a full process of processing historical market data into gym-style environments, training with the DDPG algorithm, and backtesting on a certain period of data. It showed the profitability of the DRL algorithm in stock trading. However, the results cannot imply DRL's good performance in the real market. Zhang *et al.* [22] considered both discrete and continuous action spaces and used deep Q-learning networks (DQN) [16], policy gradients (PG) [21], and advantage actor-critic (A2C) [15] in training.
- For portfolio management, Jiang *et al.* [9] presented a reinforcement manner of training a neural network with financial assets as its input and portfolio weights as its output and conducted backtest trading experiments to evaluate their results. Liang *et al.* [11] implemented three DRL algorithms, DDPG, PPO, and PG, with an adversarial training method in portfolio management and showed their promising results in backtest.
- For crypto trading, Sattarov *et al.* [18] applied DRL algorithms on the cryptocurrency market and tested the algorithms with three—Bitcoin (BTC), Litecoin (LTC), and Ethereum (ETH)—crypto coins' historical data. Sadighian *et al.* [17] proposed a DRL way to create an intelligent market making strategy, trained the agent using eight days of training data, and evaluated the agent using 30 days of testing data.

FinRL [14] provided an open-source library of financial reinforcement learning, including different DRL algorithms and multiple tasks with corresponding environments, e.g., stock trading, portfolio allocation, cryptocurrency trading, etc. However, backtesting was used in most of the implementations to show the performance of DRL agents, which is not persuasive that those agents can also perform well in real-world markets.

Those prior works applied different DRL algorithms with consideration of the market and better performance. However, their evaluations backtest using historical market data, which still has the problem of simulation to reality gap and the risk of information leakage.

There is no prior work on DRL-based paper trading, we propose a full pipeline of using DRL agent and a rolling window to train, validate and perform paper trading.

3 Deep Reinforcement Learning Approach for Stock Paper Trading

We first present an overview of stock paper trading approach by comparing it with conventional backtesting approach. Then, we describe a stock trading task and a widely used DRL algorithm, PPO algorithm [19]. Finally, we describe a “*Training-Validation-Trading*” pipeline for our stock paper trading approach.

3.1 Overview of Stock Paper Trading

In both conventional backtesting approach and our paper trading approach, historical market data are split into in-sample and out-of-sample time periods. The major difference is that backtesting may be performed multiple times, while stock paper trading is strictly required to be carried out **once** in real-time on the real-world market.

Fig. 1 (left) illustrates the conventional approach of backtesting. An agent is trained on training data, followed by validation to adapt hyperparameters (e.g., learning rate, batch size, optimizer, the total number of steps, discount factor γ). Then, the agent is retrained using all the data from the training period and validation period. Finally, one may backtest the agent’s performance during a followup backtesting time period.

Fig. 1 (right) shows the “training-validation-trading” pipeline of our stock paper trading approach. Different from the conventional backtesting approach in Fig. 1 (left), our paper trading period is always a future time, e.g., the next day. Therefore, it prohibits a potential look-ahead bias.

3.2 Problem Description of Stock Trading

MDP modeling. We model a stock trading task as a five-tuple Markov decision process (MDP), $(\mathcal{S}, \mathcal{A}, \mathbb{P}, r, \gamma)$ as follows

- State $\mathbf{s}_t = [\mathbf{p}_t, \mathbf{h}_t, b_t] \in \mathcal{S}$, where $\mathbf{p}_t \in \mathbb{R}_+^I$ denotes stock prices at t , $\mathbf{h}_t \in \mathbb{R}_+^I$ is the share holdings of stocks, and $b_t \in \mathbb{R}_+$ for the remaining balance. Here, I is the number of stocks and $\mathbb{R}_+$ denotes the set of non-negative real values.
- Action $\mathbf{a}_t \in \mathcal{A}$ denotes operations on the D stocks. Available actions include {buying, selling, holding}.
- Transition probability $\mathbb{P}(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ describes the unknown market environment, which is the probability of transiting to state $\mathbf{s}'$ when taking action $\mathbf{a}$ at state $\mathbf{s}$.
- Reward function $r(\mathbf{s}_t, \mathbf{a}_t)$ is chosen to be the change of portfolio value, $v' - v$, where v' and v denotes the portfolio value at state $\mathbf{s}_{t+1}$ and $\mathbf{s}_t$, respectively.
- $\gamma \in [0, 1]$ is a discount factor.

A policy $\pi(\mathbf{s})$ is a probability distribution over the actions at state $\mathbf{s}$, which is reparameterized as a neural network.

We follow the OpenAI gym style [3] to build a stock market environment by replaying the historical data. The DRL agent interacts with the environment iteratively for T total steps. For each time step t , the agent takes an action $\mathbf{a}_t$ on $\mathbf{s}_t$ and send it to the environment. The environment receives and executes $\mathbf{a}_t$ and send back a new state $\mathbf{s}_{t+1}$ with a reward r_t . The agent stops its action when it reaches the terminal state $\mathbf{s}_{T-1}$.

Market environment. The stock market environment has the following three functions:

- **Reset($\cdot$)** function resets $\mathbf{s} \in \mathcal{S}$ to an initial condition such that $\mathcal{S} = [\mathbf{p}_0, \mathbf{h}_0, b_0]$ where $\mathbf{h}_0$ is a zero vector since there is no share holdings in the beginning and b_0 is the initial investment capital.
- **Step($\cdot$)** function executes an action $\mathbf{a}_t$ and updates state $\mathbf{s}_t$ to $\mathbf{s}_{t+1}$ accordingly. For the transition from state $\mathbf{s}_t$ to $\mathbf{s}_{t+1}$, $\mathbf{p}_{t+1}$ is accessible from the market data; $\mathbf{h}_{t+1}$ and b_{t+1} are updated as $\mathbf{h}_{t+1} = \mathbf{h}_t + \mathbf{a}_t$ and $b_{t+1} = b_t - \mathbf{p}_t^T \mathbf{a}_t$ respectively, where negative actions mean selling and positive actions mean buying.

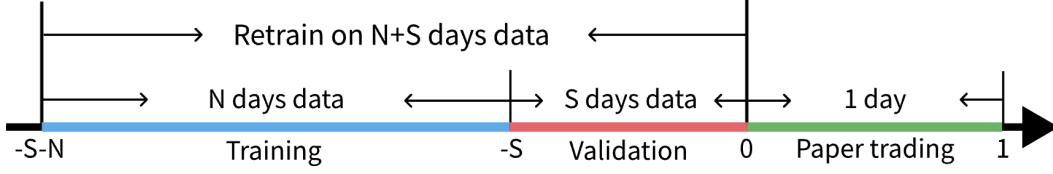


Figure 2: Data split for a window with training, validation and trading.

Algorithm 1 Algorithm for stock paper trading

```

1: Initialize a set of hyper-parameters;
2: for  $d = 0$  to  $D - 1$  do
3:   # Step 1). Train an agent
4:   Using data period  $[d - S - N, d - S - 1]$  to train an agent,
5:   Using data period  $[d - S, d - 1]$  to validate the trained agent and adjust hyper-parameters,
6:   # Step 2). Retrain the agent
7:   Using data period  $[d - S - N, d - 1]$  to retrain the agent with the adjusted hyper-parameters,
8:   # Step 3). Perform paper trading for one day
9:   Using the trained agent to trade in the  $d$ -th day.
10: end for

```

- **Reward**($\cdot$) function returns the value of $r(s_t, a_t, s_{t+1})$, which is $v_{t+1} - v_t$, computed as $r(s_t, a_t, s_{t+1}) = (b_{t+1} + p_{t+1}^T h_{t+1}) - (b_t + p_t^T h_t)$.

3.3 Proximal Policy Optimization (PPO) Algorithm

We select the Proximal Policy Optimization (PPO) algorithm [19] to maximize the cumulative return of paper trading. PPO is trying to make the biggest possible update while keeping the new policy close to the old one at the same time. It is shown that PPO has great performance in many different kinds of tasks. Actor-critic approach on stock trading has been shown effective in [13]. Instead of DDPG, here we would use on policy algorithm PPO.

PPO algorithm updates a policy with parameter θ by $\theta_{k+1} = \arg\max_{\theta} \mathcal{L}_{\theta_k}(\theta)$, where the objective function $\mathcal{L}_{\theta_k}(\theta)$ is defined as follows:

$$\mathcal{L}_{\theta_k}(\theta) = \mathbb{E}_{\mathcal{T} \sim \pi_k} \left[\sum \min\left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \text{clip}\left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon\right) A^{\pi_{\theta_k}}(s_t, a_t)\right) \right], \quad (1)$$

where $\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$ is a ratio between the new and old policy networks, $A^{\pi_{\theta_k}}(s_t, a_t)$ is the advantage function, and ϵ is a hyperparameter to decide how different the new and old policies can be. If the probability ratio between the new and old policies falls outside the range $(1 - \epsilon)$ and $(1 + \epsilon)$, the advantage function will be clipped to $1 - \epsilon$ or $1 + \epsilon$, respectively.

The two policy networks, $\pi_{\theta}(a_t|s_t)$ and $\pi_{\theta_k}(a_t|s_t)$ represent the new and old policy networks. As we are updating the policy iteratively, the difference between the two policies can get larger, resulting in bad decisions of the agent. Thus, we synchronize the old policy with a updated policy for every n iterations as $\pi_{\theta_{k+1}}(a_t|s_t) = \pi_{\theta}(a_t|s_t)$.

3.4 "Training-Validation-Trading" Pipeline of Paper Trading Task

Fig. 2 shows our "training-validation-trading" pipeline. In a window, there are N days' data for training and S days' data for validation. At the end of a window, we perform paper trading for 1 day. Note that we always retrain the agent using $N + S$ days of training and validation data together. Then, we roll the window forward by 1 day ahead and perform the above steps for a new window. A paper trading is always carried out for 1 day. Therefore, D windows correspond to D trading days.

Alg. 1 summarizes the pipeline of paper trading. For D trading days from 0 to $D - 1$, we keep doing the following three steps:

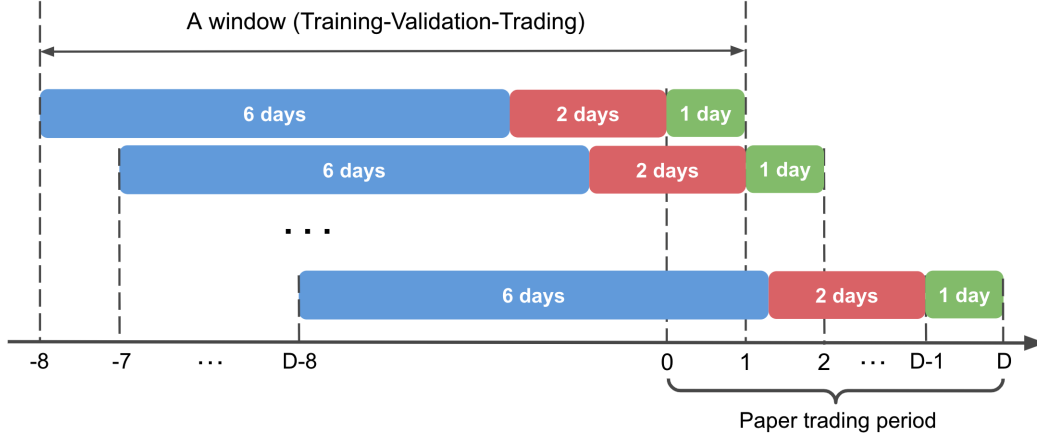


Figure 3: The rolling windows used in our experiments.

- **Step 1).** Download and process N -day data, from day $d - S - N$ to day $d - S - 1$. Then build the data into a gym-style environment and train the agent. Then download and process S -day data, from day $d - S$ to day $d - 1$. Then build the data into a gym-style environment and validate how the agent performs. According to the agent's performance on the validation environment, adjust hyper-parameters.
- **Step 2).** Build the training and validation data, totally $N + S$ days from day $d - S - N$ to day $d - 1$ (note that there are 390 data points for each day's minute level data), into a gym-style environment. Update hyper-parameters to the values chosen from **Step 1).** Then retrain the agent on these $N + S$ -day environment.
- **Step 3).** Deploy the trained agent to the paper trading market and trade from 9:30am to 4:00pm.

Before each trading day d , we use the historical data period from $d - S - N$ to $d - S - 1$ to train the DRL agent. Then, data periods from $d - S$ to $d - 1$ are used to validate the trained agent, adjusting hyper-parameters accordingly. We retrain the agent by combining all the data from the training and validation period, that is, the data period from $d - S - N$ to $d - 1$. Finally, we perform 1 day paper trading on the d -th day. We continue this process for $d = 0, 1, \dots, D - 1$.

Fig. 2 presents the mechanism of each rolling window. In our experiments, there are in total 9 days in a rolling window. We divide it into three stages, namely, $N = 6$ days of training, $S = 2$ days of validation, and 1 day of paper trading using the trained DRL agent. Note that we retrain our agents for $N + S = 8$ days before each trading day. We train the agent with the DRL algorithm in the training stage, using a set of hyper-parameters. The validation stage allows us to adjust the hyper-parameters, such as learning rate and batch size. Then, we train our agents again using the data from the training and validation stage. Finally, we use the well-trained agent to perform paper trading on the last day of a rolling window.

4 Performance Evaluations

We empirically evaluate the trading performance of the PPO algorithm for a stock paper trading task. We compare it with a conventional machine learning method (random forest [2]) and the Dow Jones Industrial Average (DJIA) index (market benchmark performance).

4.1 Experimental Settings

We select Dow Jones 30 stocks as our trading stocks and use minute-level historical market data from 08/24/2022 to 09/09/2022. The data are downloaded from Alpaca¹. Then, we use the paper trading APIs provided by Alpaca to do paper trading from 09/01/2022 to 09/12/2022.

¹Web page of Alpaca: <https://alpaca.markets/>

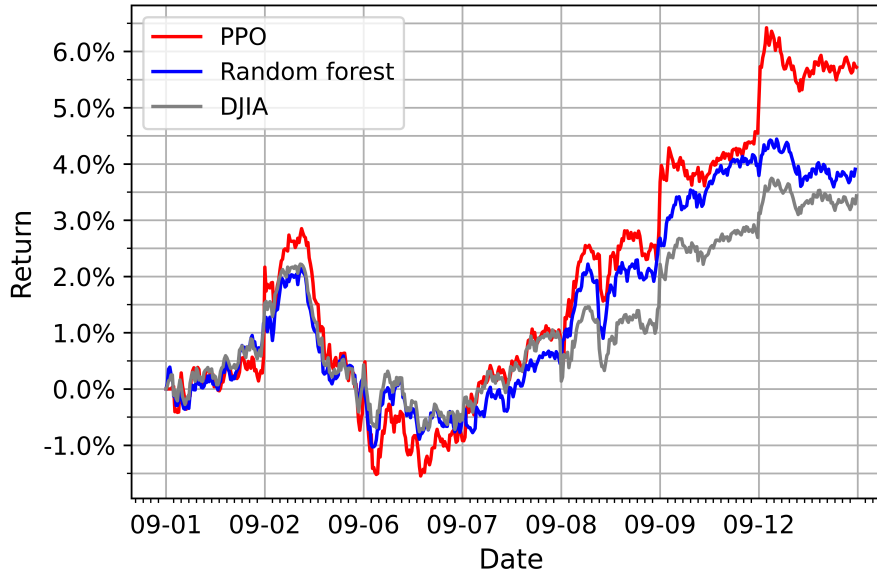


Figure 4: Cumulative returns: a conventional ML method (random forest) VS. our trained PPO agent.

The first step is to download and process the data into a market environment as described in Section 3.2, following the time split of the rolling window in Fig. 3. We build the market environment by organizing the 30 stocks' prices per minute into a vector. A PPO agent is trained on the environment over a 6 days' minute-level data, i.e., there are $6 \times 390 = 2,340$ data points. Then, we validate the trained PPO agent on 2 days' minute-level data, and tune hyper-parameters according to the performance. After that, we retrain the agent on 8 days' minute-level data. Finally, we deploy our agent on paper trading from 9:30 am to 4:00 pm in real-time.

Our experiment has three stages, namely training, validation, and trading. In the training stage, the DRL algorithm generates a well-trained trading agent. The validation stage is then carried out for key parameters adjustment such as learning rate, number of episodes, batch size, etc. Then the agent is retrained before trading. Finally, we perform paper trading on the Alpaca platform using the trained agent and evaluate its performance in the trading stage.

Fig. 3 illustrates the rolling windows used in our experiments. Each window consists of $N + S + 1 = 9$ days, where $N = 6$ and $S = 2$. We performed paper trading for $D = 7$ days, starting on 09/01/2022 (day 0) and ending on 09/12/2022 (day $D - 1$). We followed the "training-validation-trading pipeline, thus our agent always learns the latest market data without look-ahead bias.

4.2 Trading Performance

Performance metrics: We compare the performance of our PPO approach to that of the random forest approach and Dow Jones Industrial Average (DJIA). Six metrics are chosen to evaluate the trading performance: final portfolio value, cumulative return, average daily return, standard deviation of daily return, Maximum drawdown, and Sharpe ratio. The final portfolio value reflects portfolio value at the end of the trading stage. The average daily return shows an average value of the direct return of the portfolio per day. Daily standard error indicates the robustness of our model. Sharpe Ratio represents the expected return relative to the unit amount of investment risk.

Fig. 4 shows the trading performance during the period of Sep. 1st to 12nd, 2022. Compared with the DJIA index, we can see that both random forest method and our PPO agent have excess returns. Our PPO strategy has evident advantage on the final return than the others. However, when the prices are falling, our PPO strategy has a larger decline. We conclude that our PPO agent performs better than the random forest method and DJIA in terms of cumulative return, but has a larger fluctuation.

Table 1: Trading performance.

	PPO (ours)	Random Forest [2]	DJIA Index
Initial portfolio value	100,000	100,000	100,000
Final portfolio value	105,720	103,906	103,433
Cumulative return	5.72%	3.906%	3.433%
Avg. daily return	0.82%	0.56%	0.49%
Std. of daily return	2.33%	1.72%	1.27%
Max. drawdown	-4.27%	-3.10%	-2.91%
Sharpe ratio	0.82	0.96	1.02

Table 1 lists the values for different trading metrics. Using the same initial capital of \$100,000, our PPO strategy achieves an final portfolio value of 105,720, which is significantly higher than random forest’s 103,906 and the DJIA index’s 103,433. In addition to that, our PPO strategy has significant advantage on cumulative return and average daily return. However, it also has the largest standard deviation of daily return and maximum drawdown among the three strategies. This implies our PPO strategy is more unstable and easier to be influenced by the fluctuation of market. Moreover, our PPO agent has the lowest sharpe ratio, which means its expected return relative to the same level of risk is lower than the other two.

5 Conclusion and Future Work

In this paper, we have empirically demonstrated the potential of deep reinforcement learning (DRL) for a paper trading task. Compared with existing backtesting results that may suffer look-ahead bias and overfitting, our stock paper trading approach is one step closer to the real-world markets. The results show that a trained PPO agent outperforms the Dow Jones Industrial Average index and a conventional machine learning method. We believe that a DRL agent that performs well in paper trading is more likely to profit in a live trading task.

In future works, we would like to provide verifications of other DRL algorithms, such as deep deterministic policy gradient (DDPG) [12], soft actor-critic (SAC), etc. We would try different sizes of rolling windows; for example, change the data frequency to 1 hour level or 1 second level. Also, it is interesting to benchmark the profitability of DRL for various paper trading tasks, e.g., stock trading, portfolio management, crypto trading, etc.

References

- [1] David H Bailey, Jonathan Borwein, Marcos Lopez de Prado, Amir Salehipour, and Qiji Jim Zhu. Backtest overfitting in financial markets. *Automated Trader*, 2016.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [4] Hans Buehler, Lukas Gonon, Josef Teichmann, and Ben Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.
- [5] Marcos Lopez De Prado. *Advances in financial machine learning*. John Wiley & Sons, 2018.
- [6] Thomas G Fischer. Reinforcement learning in financial markets-a survey. Technical report, FAU Discussion Papers in Economics, 2018.
- [7] M Ugur Gudelek, S Arda Boluk, and A Murat Ozbayoglu. A deep learning based stock trading model with 2-d cnn trend detection. In *IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1–8. IEEE, 2017.
- [8] Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *arXiv preprint arXiv:2112.04553*, 2021.
- [9] Zhengyao Jiang and Jinjun Liang. Cryptocurrency portfolio management with deep reinforcement learning. In *Intelligent Systems Conference (IntelliSys)*, pages 905–913. IEEE, 2017.
- [10] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.
- [11] Zhipeng Liang, Hao Chen, Junhao Zhu, Kangkang Jiang, and Yanran Li. Adversarial deep reinforcement learning in portfolio management. *arXiv preprint arXiv:1808.09940*, 2018.
- [12] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *ICLR (Poster)*, 2016.
- [13] Xiao-Yang Liu, Zhuoran Xiong, Shan Zhong, Hongyang Yang, and Anwar Walid. Practical deep reinforcement learning approach for stock trading. *NeurIPS Workshop on Challenges and Opportunities for AI in Financial Services: the Impact of Fairness, Explainability, Accuracy, and Privacy*, 2018.
- [14] Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *NeurIPS Workshop on Deep Reinforcement Learning*, 2020.
- [15] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.
- [16] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [17] Jonathan Sadighian. Extending deep reinforcement learning frameworks in cryptocurrency market making. *arXiv preprint arXiv:2004.06985*, 2020.
- [18] Otabek Sattarov, Azamjon Muminov, Cheol Won Lee, Hyun Kyu Kang, Ryumduck Oh, Junho Ahn, Hyung Jun Oh, and Heung Seok Jeon. Recommending cryptocurrency trading points with deep reinforcement learning approach. *Applied Sciences*, 10(4):1506, 2020.
- [19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- [20] Tong Sun, Jia Wang, Jing Ni, Yu Cao, and Benyuan Liu. Predicting futures market movement using deep neural networks. In *IEEE International Conference On Machine Learning And Applications (ICMLA)*, pages 118–125. IEEE, 2019.
- [21] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.
- [22] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2(2):25–40, 2020.